2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design (CSCWD) | 978-1-7281-6597-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/CSCWD49262.2021.9437719

Cloud-Based Lightweight Collaborative Editing Algorithm for Mobile Devices

Xiao Lv[1], Jiacun Yuan[1], Fazhi He[2], Yuan Cheng[3], Weiwei Cai[2]
[1] College of Computer Engineering
Naval University of Engineering, Wuhan, P.R.China 430033
Email:XiaoLv_nue@163.com
[2] School of Computer Science and Technology
Wuhan University, Wuhan, P.R.China 430072
[3] School of Information Management
Wuhan University, Wuhan, P.R.China 430072

***Abstract*—With the burgeoning of mobile devices, collaborative editing through mobile devices has become an alternative choice to traditional desktop PCs. However, mobile collaborative editing has to face some technical challenges. How to achieve an efficient mobile collaboration while maintaining the shared document copies consistency is one of the core issues in collaborative computing area. This paper proposes a cloud-based lightweight collaborative algorithm for mobile devices. Firstly, a cloud-based mobile collaborative editing framework is proposed for collaborating between cloud platform and mobile devices. Secondly, a collaborative editing task allocation model is constructed to assign editing operations for cloud platform and mobile devices. Thirdly, a cloud-based lightweight collaborative editing approach is presented to achieve the data synchronization and the consistency maintenance for the shared document copies. Finally, experiments evaluations prove the effectiveness of the proposed approach.**

***Index Terms*—mobile collaborative editing, operational transformation (OT), commutative replicated data type (CRDT), consistency maintenance, data synchronization**

I. Introduction

Nowadays, a variety of mobile collaboration applications have been developed and support people's daily work, study and life [1]–[5]. As a typical representative, mobile collaborative editing systems permit a great number of geographically dispersed mobile users to edit a shared document text [6], xml [7], images [8], spreadsheet [9] and other types of document [10]–[12] from anywhere at any time through mobile networks. Mobile collaborative editing is opening a new way for the next generation of collaborative editing systems.

In order to attract and retain more mobile users, the responsiveness time is the main factor of success for mobile collaboration. Nevertheless, some new technical challenges need to be addressed. Firstly, it is challenging to provide an efficient computing performance and less energy consumption for mobile devices. Secondly, it is hard to maintain a persistent collaboration due to unstable mobile networks. Therefore, a more advanced collaborative control mechanism is needed to allow mobile users to edit the shared document copies in an online or offline mode. Thirdly, it is more sophisticated to maintain the document consistency because online or offline document copies need to be synchronized continuously.

In order to address the above challenges, this paper presents a cloud-based lightweight collaborative editing algorithm for mobile devices. Firstly, a cloud-based mobile collaborative editing framework and a task allocation model are proposed for collaborating between cloud platform and mobile devices. Furthermore, a cloud-based mobile collaborative editing approach is presented to achieve the data synchronization and the consistency maintenance. Finally, experiments evaluations prove the effectiveness of the proposed framework, model and approach.

II. Background and Related Work

Over the past 30 years, a lot of researches have been dedicated to collaborative editing systems [12]–[16]. Some typical collaborative applications have been applied in real-world study and work, such as Google Wave [1], Google Docs [2], Dropbox [3], Box Notes [4]. The mainstream consistency maintenance techniques of collaborative editing include OT (Operational Transformation), AST (Address Space Transformation) and CRDT (commutative replicated data type).

OT is the first collaborative editing algorithm which has been proposed to maintain the consistency of the shared document. A plethora of OT algorithms have been presented for collaborative editing systems [9], [14], [15], [17]. Over the past decade, the scope of OT research has evolved from plain-text collaborative editing to a variety of collaborative office applications, including word processors [6], [18], HTML/XML Web document [19], spreadsheet [9], 3D digital media design [17] and file synchronization in cloud storage [20]. Currently, several OT-based algorithms have been developed for mobile collaboration, such as SPOC [1], CloneDoc [2], Google Docs [21] and OptiCloud [4].

This work was supported by National Key R&D Program of China (Grant No.2017YFB0503004), National Natural Science Foundation of China (Grant No.61502353) and Natural Science Foundation of Hubei Province (No.2019CFB627).

[1]http://www.waveprotocol.org/whitepapers/operational- transform
[2]https://docs.google.com
[3]https://paper.dropbox.com/
[4]https://www.box.com/notes

Authorized licensed use limited to: Southwest University. Downloaded on April 04,2026 at 08:58:37 UTC from IEEE Xplore. Restrictions apply.

AST is one notable class of consistency maintenance techniques for supporting collaborative editing without operational transformation. The first work was proposed in 2005 [22], then several AST-based consistency maintenance algorithms have been presented for collaborative editing systems [3], [23]–[25]. Recently, Hermes [3] uses AST-based technique to provide heterogeneous collaborative editing services for mobile users.

CRDT is an alternative consistency maintenance technique for supporting real-time collaborative editing systems [12], [16], [26]–[31]. Since the first CRDT work proposed by Oster in 2006, a sequence of CRDT-based algorithms have been developed for collaborative editing systems. In recent years, CRDT is one prominent class of techniques for collaborative text editing systems. GitHub [5] has published some typical CRDT-based collaborative editing products, such as Teletype [6] and Alchemy Book [7]. However, the above discussed products adopt the central server to implement some features of collaborative editing systems [32].

To our best knowledge, CRDT-based collaborative editing methods for cloud-based mobile collaboration are rarely investigated in existing published researches. How well CRDT-based technique for cloud-based mobile collaboration could perform is still unknown. To get an idea, this paper designs an efficient CRDT-based lightweight collaborative editing algorithm for the cloud and the mobile device. However, it is not an easy work to provide an efficient computing performance with the limited resources of mobile devices while maintaining the consistency of the document copies. Moreover, a more advanced data synchronization mechanism and consistency maintenance approach are needed to maintain the consistent collaborative results on mobile devices.

III. Cloud-based Lightweight Mobile Collaborative Editing Algorithm

A. Relate Definitions

Definition 1: An operation o is a six-tuple $< type, l, tar_key, cur_key, con, tar_o >$ where
(1) $type$ is an enum variable which shows the type of an operation o.
(2) l is a variable that represents local or remote operations.
(3) tar_key is the identifier of the target operation.
(4) cur_key is the identifier of the current operation.
(5) con is the content of the inserted or deleted block.
(6) tar_o is the target operation to be undone.

Definition 2: The identifier of o is a three-tuple $< s, site, ssv >$ where
(1) s is the identifier of a session, which is a global increasing number.
(2) $site$ is the unique identifier of the site.
(3) ssv is the sum of state vector of an operation.

[5]https://github.com
[6]https://github.com/atom/teletype
[7]https://github.com/rudi-c/alchemy-book

Definition 3: An operation object $block$ is a five-tuple $< cur_key, visible, con, next, link >$ where
(1) cur_key is the identifier of each block.
(2) $visible$ is an integer variable that its value can be positive, negative or 0.
(3) con is the content of the block, which may be the atomic character or unbreakable string.
(4) $next$ is a pointer, which points the next block in the single linked list ($List$).
(5) $link$ is a pointer, which is used for the double chaining in the hash table ($Hash\ Table$).

Each operation refers to the responding block. Each block represents the inserted (deleted) object of the operation. The detail illustrations of Definition 1 to Definition 3 are shown in our previous work [30].

Definition 4: $block\prec$
Given any two blocks in $List$, $block_i$ and $block_j$. The positions of $block_i$ and $block_j$ are denoted as p ($block_i$) and p ($block_j$). If $block_i \prec block_j$, iff: p ($block_i$)< p ($block_j$) ($block_i$ is before $block_j$ in $List$).

Definition 5: Total ordering relations of blocks($block\Rightarrow$)
Given any three blocks, $block_i$, $block_j$ and $block_k$, then $block\Rightarrow$ exists, iff:
(1) $block_i \prec block_j$, or $block_j \prec block_i$, or
(2) $block_i \prec block_j$, $block_j \prec block_k$, then $block_i \prec block_k$.

Definition 6: The intention of an operation o
Assume that $block$ is the object of o, the operation intention of o is defined as the position relation of $block$ in $List$.

Definition 7: Consistency of operation intentions
Assume that $List_1$, $List_2$,...,$List_m$ are respectively maintained by m mobile devices. Suppose that there are n blocks in each $List$. For any two $Lists$, $List_i$ and $List_j$, $block \overset{List_i}{\Rightarrow}$ ($block \overset{List_j}{\Rightarrow}$) represents the total ordering relations of $List_i$ ($List_j$). When all operations have been executed in m sites, consistency of operation intentions is achieved iff: $block \overset{List_i}{\Rightarrow} = block \overset{List_j}{\Rightarrow}$.

B. Cloud-based collaborative editing framework

Fig. 1 shows an overall cloud-based collaborative editing framework, which consists of multiple front-end mobile devices and the back-end cloud platform. Each mobile device maintains a local copy of the shared document (DS) and some local operation logs, $locallog$, $remotelog$ and log. The detail data structures of DS have been presented in our previous work [30]. Local operations generated by mobile devices are stored in $locallog$. Received transformed remote operations are retained in $remotelog$. All executed local and remote operations on mobile devices are stored in log.

The cloud platform maintains all clones of their front-end mobile devices. Each clone maintains DS', $locallog'$, $remotelog'$, $remotelog''$, $c2mremotelog$ and log'. DS' is the copy of its front-end mobile device' DS. Local operations from mobile devices are stored in $locallog'$. Remote operations from other clones are retained in $remotelog'$. Executed remote operations are stored in $remotelog''$. Transformed remote

Authorized licensed use limited to: Southwest University. Downloaded on April 04,2026 at 08:58:37 UTC from IEEE Xplore. Restrictions apply.

operations that will be synchronized with mobile devices are retained in $c2mremotelog$. All executed local and remote operations are stored in log'. Cloud-based mobile collaborative editing module has two main functions. Firstly, it provides a distributed data synchronization mechanism between back-end clones in cloud platform and front-end mobile devices. Secondly, it provides the consistency maintenance technique for all clones' copies of the shared document in cloud platform.

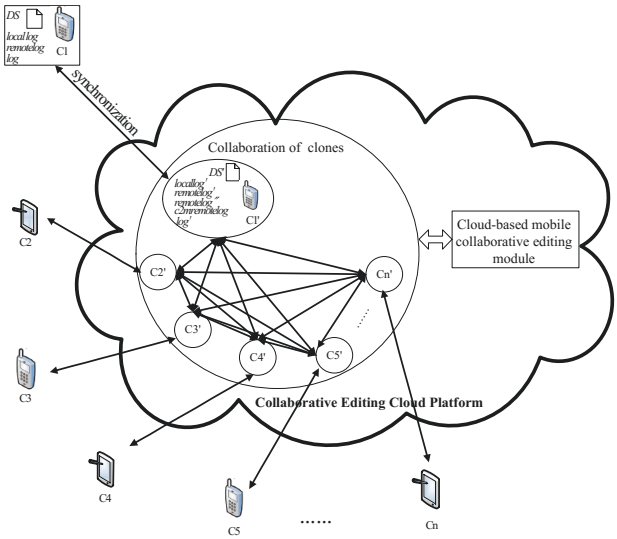

Fig. 1. Cloud-based mobile collaborative editing framework.

C. Collaborative editing task allocation model

Fig. 2 shows the collaborative editing task allocation model between mobile devices and clones in cloud platform. For mobile devices, they generate and integrate local operations, and execute transformed remote operations. For clones in cloud platform, they integrate local operations from mobile devices, transform and integrate remote operations from other clones, and generate executable remote operations for mobile devices.

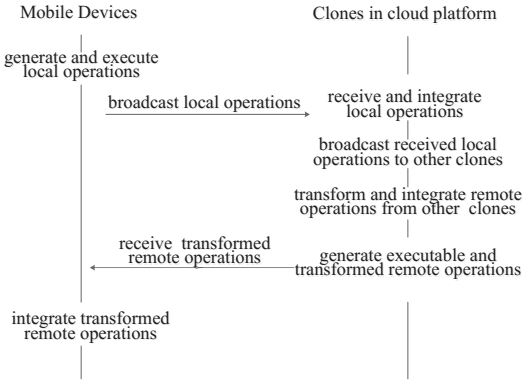

Fig. 2. Cloud-based collaborative editing task allocation model.

D. Cloud-based mobile collaborative editing algorithm

Algorithm 1 presents the processing procedure of collaborative editing for mobile devices. In line 4, a synchronization variable m is set to 1, which shows that mobile devices are ready to generate and integrate local operations. In line 7, local operations are integrated by invoking the function IntegrateLocal (o), which has been presented in our previous work [30]. In line 11, m is set to 0, which shows that mobile devices are ready to synchronize with their clones in cloud. In lines 12-15, local operations are broadcast to their clones and transformed remote operations are received by mobile devices. In lines 16-19, the operation effect of each remote operation is directly applied in DS.

Algorithm 1 MobileCollaborativeEdit

1: **Initialization_Mobile:**
2: DS = Doc
3: $locallog$=null, $remotelog$=null, log=null
4: int m=1
5: **Generation_Integration_Local:**
6: **if** m==1 **then**
7: DS=IntegrateLocal(o)
8: $locallog$=Add(o,$locallog$),log=Add(o,log)
9: **end if**
10: **Synchronization_With_Clone:**
11: m=0
12: send $locallog$ to its clone
13: $locallog$.clear( )
14: receive $c2mremotelog$ from its clone
15: $remotelog$=$c2mremotelog$
16: **for** each o' in $remotelog$ **do**
17: DS= Apply(o',DS)
18: Log=Add(o',log)
19: **end for**
20: $remotelog$.clear()
21: m=1

Algorithm 2 presents the processing procedure of collaborative editing for clones in cloud. In line 4, a synchronization variable c is set to 0, which represents that clones in cloud are ready to synchronize with their mobile devices. In lines 9-13, local operations are integrated and the executed remote operations are transformed to correct forms. In line 10, the function OpExecution (o_l) is invoked to generate the correct executable operation o_l'. In line 11, the operation effect of o_l' is directly applied in DS'. In line 12, the function TransformRemote (o_l') is called to get the transformed remote operation because the execution of o_l' may change the operation position of an executed remote operation. In lines 15-20, all transformed remote operations are added to $c2mremotelog$ and broadcast to mobile devices. In lines 23-34, remote operations from other clones are integrated in DS'.

Authorized licensed use limited to: Southwest University. Downloaded on April 04,2026 at 08:58:37 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 CloneCollaborativeEdit

Initialization_Clone:
$DS' = Doc$
$locallog'$=null,$remotelog'$=null,$remotelog''$=null
$c2mremotelog$=null,log'=null, int c=0
Synchronization_With_Mobile:
if c==0 **then**
  receive $locallog$ from mobile
  $locallog' = locallog$
  for each o_l in $locallog'$ **do**
    o_l'=OpExecution(o_l)
    DS'=Apply(o_l',DS'),log'=Add(o_l',log')
    TransformRemote(o_l')
  end for
  $locallog'$.clear()
  for each o_r' in $remotelog''$ **do**
    $c2mremotelog$ = Add(o_r', $c2mremotelog$)
  end for
  $remotelog''$.clear()
  send $c2mremotelog$ to its mobile device
  $c2mremotelog$.clear()
  c=1
end if
Integration_Remote_From_Clone:
if $c = 1$ **then**
  for each o_r in $remotelog'$ **do**
    o_r'=OpExecution(o_r)
    DS'=Apply(o_r',DS')
    log'=Add(o_r',log')
    $remotelog''$=Add(o_r',$remotelog''$)
    TransformRemote(o_r')
  end for
  $remotelog'$.clear()
  $c = 0$
end if

Algorithm 3 OpExecution(o)

1: $tar_key = o.tar_key$
2: **if** $o.type == insertion$ **then**
3: $Tar_block = hash(o.tar_key)$
4: **if** Tar_block is $head$ **then**
5: $pre_block = head$
6: **else**
7: $pre_block = Tar_block$
8: **end if**
9: $next_block = pre_block.next$
10: **while** $next_block$! = null **do**
11: **if** $o.cur_key > next_block.cur_key$ **then**
12: break
13: **else**
14: $pre_block = next_block$
15: $next_block = next_block.next$
16: **end if**
17: **end while**
18: $tar_key = pre_block.cur_key$
19: **end if**
20: $o.tar_key = tar_key$

Algorithm 4 TransformRemote(o)

1: **if** $o.type == insertion$ **then**
2: $New_block = hash(o.cur_key)$
3: $Next_block = New_block.next$
4: **if** $Next_block.l == 0$ **then**
5: o_n = GetOp($Next_block.cur_key$, $remotelog''$)
6: $o_n.tar_key = o.cur_key$
7: **end if**
8: **end if**

Algorithm 3 presents the processing procedure of the function OpExecution (o). The parameter o is a local operation from mobile or a remote operation from other clones in the cloud. In lines 2-19, if o is an insertion, the target block of o needs to be found by comparing the identifiers of concurrent operations. In line 3, the target block of o (Tar_block) is found in $List$. In lines 10-17, some concurrent insertions may have already inserted their blocks next to Tar_block. Therefore, it is necessary to scan the blocks next to Tar_block until a block that its $cur_key < o.cur_key$ is first encountered. In line 20, tar_key of o is updated by $pre_block.cur_key$.

Algorithm 4 presents the processing procedure of the function TransformRemote (o). The parameter o is an executed local or an executed remote operation in clones. In lines 2-3, a new inserted block of o (New_block) and its next block ($Next_block$) are found in $List$. If $Next_block$ is an object of an executed remote operation (o_n), o_n is found from $remotelog''$ (Lines 4-5). Because New_block has been inserted before the object of o_n, the identifier of o is as the target identifier of o_n (Line 6).

IV. Experiments Evaluations

A. Implementation of cloud-based mobile collaboration

We develop and implement mobile collaboration and cloud collaboration to evaluate the performance of the proposed approach. Mobile collaboration is developed on JAVA language and compiled in Android Studio on Windows 10 system, intel (R) core (TM) i7-9750H, 2.60 GHz CPU. Cloud collaboration is developed on JAVA language and compiled in Eclipse and Tomcat web server. We select the android device as the test device and install the developed mobile collaboration application on it.

Here, we more focus on the responsiveness time and energy consumption of integrating operations on mobile devices. In experiment evaluations, we assume that m android mobiles ($m > 2$) involve in the collaboration and all android devices are connected with Wi-Fi communication. We design collaboration procedures to obtain operation logs and replay all operations on mobile devices. We run the proposed approach for 20 times in each operation log and the average responsiveness time and the energy consumption are recorded.

Authorized licensed use limited to: Southwest University. Downloaded on April 04,2026 at 08:58:37 UTC from IEEE Xplore. Restrictions apply.

B. Performance Evaluation

1) Responsiveness Time: We select HONOR PCT-AL10 as the test device. The responsiveness time mainly involves the required time for an operation integrated and synchronized in a mobile device.

Fig. 3 (a) shows the average responsiveness time of integrating a normal operation from mobile. We can observe that the responsiveness time of a remote operation increases in a large scale with the growth of the number of operations. The main reason is that the mobile device spends more time to compute the correct inserted position by scanning some concurrent blocks. Fig. 3 (b) shows the average responsiveness time of integrating a remote operation from mobile and a transformed remote operation from cloud. We notice that the responsiveness time of integrating a transformed remote operation from cloud is much lower. The reason is that mobile devices directly execute transformed remote operations without concurrency conflict resolution due to the computation and transformation of the cloud.

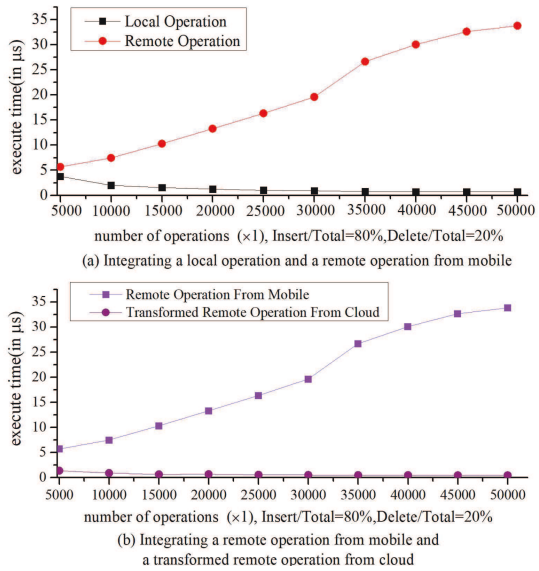

Fig. 3. The average responsiveness time of integrating an operation.

2) Energy Consumption: We use the ADB (Android Debug Bridge) tool in Android Studio to test the energy consumption. We test the energy consumption for mobile to mobile collaboration and mobile to cloud collaboration.

Fig. 4 (a) shows the energy consumption of integrating different numbers of normal operations from mobile. We can observe that integrating remote operations in mobile devices consumes more energy since remote operations must find the correct operation position by scanning some concurrent inserted blocks. Fig. 4 (b) shows the energy consumption of integrating remote operations from mobile and transformed remote operations from cloud. We notice that integrating transformed remote operations from cloud consumes less energy since the cloud has computed the correct operation positions for mobile devices.

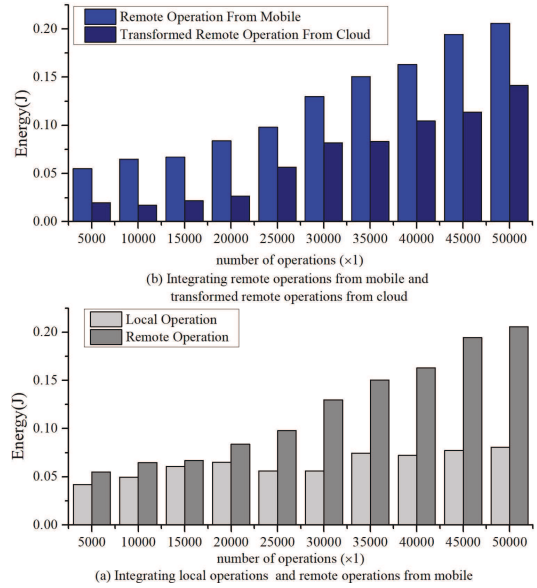

Fig. 4. Energy consumption of integrating operations.

V. Comparison with Previous Work

Early several concurrency control algorithms have been proposed for distributed DBMS (DDBMS) [33]–[35]. Concurrency control algorithms based on transaction processing generally have the complication of control mechanism and the cost to response time [14], which are not well suitable for mobile collaboration.

Currently, some concurreny control algorithms have been developed for mobile collaboration [1]–[4], [21]. SPOC system uses OT-based technique to store, share, and transform operations [1]. CloneDoc system adopts OT-based technique to transform and execute operations on mobile devices [2]. Hermes [3] uses AST-based technique to provide collaborative editing services for heterogeneous mobile users. OptiCloud uses OT-based technique to preserve the consistency of mobile to clone and clone to clone in cloud [4]. Google Docs broadcasts all operations to a central server and sequences all operations with first in first out mechanism [21].

Different from the above discussed approaches, this paper adopts CRDT-based technique to preserve the consistency of the shared document copies. To our best knowledge, CRDT-based technique for cloud-based mobile collaboration is rarely investigated in existing published researches. The most related to this paper is our previous work [30], which only supports do and selective undo of unbreakable strings for traditional desktop PCs. Different from the literature [30], this paper supports cloud-based mobile collaboration. Firstly, the proposed approach needs to maintain some new data structures $locallog'$, $remotelog'$, $remotelog''$, $c2mremotelog$ and log' in addition to DS presented in [30]. Secondly, this paper presents a new task allocation model to assign lightweight computation to mobile devices and most of computation to the cloud. Thirdly, this paper proposes an online or offline data synchronization and consistency maintenance mechanism.

Authorized licensed use limited to: Southwest University. Downloaded on April 04,2026 at 08:58:37 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusions and Future Work

This paper presents a cloud-based mobile collaborative editing framework and a task allocation model for collaborating between cloud platform and mobile devices. Furthermore, a cloud-based mobile collaborative editing approach is presented to achieve the data synchronization and the consistency maintenance. Experiments evaluations prove the effectiveness of the proposed approach. In future work, we plan to extend this work to ot collaborative applications in Images/Graphics/CAD [36]–[38].

Acknowledgment

This work was supported by National Key R&D Program of China (Grant No.2017YFB0503004), National Natural Science Foundation of China (Grant No.61502353) and Natural Science Foundation of Hubei Province (No.2019CFB627).

References

[1] A. J. Feldman, W. P. Zeller, M. J. Freedman, E. W. Felten, Sporc: group collaboration using untrusted cloud resources, in Proceedings of 9th USENIX Symposium on Operating Systems Design and Implementation, ACM, 2010, pp. 337–350.

[2] S. Kosta, V. C. Perta, J. Stefa, P. Hui, A. Mei, Clonedoc: exploiting the cloud to leverage secure group collaboration mechanisms for smartphones, in Proceedings of IEEE Conference on Computer Communications Workshops, IEEE, 2013, pp. 19–20.

[3] H. Xia, T. Lu, B. Shao, X. Ding, N. Gu, Hermes: On collaboration across heterogeneous collaborative editing services in the cloud, in Proceedings of International Conference on Computer Supported Cooperative Work in Design, IEEE, 2014, pp. 655–660.

[4] M. D. Mechaoui, N. Guetmi, A. Imine, Mica : Lightweight and mobile collaboration across a collaborative editing service in the cloud, Peer-to-peer Networking and Applications 9 (6) (2016) 1242–1269.

[5] S. Franz, R. Irmler, U. Rüppel, Real-time collaborative reconstruction of digital building models with mobile devices, Advanced Engineering Informatics 38 (OCT.) (2018) 569–580.

[6] S. Xia, D. Sun, C. Sun, D. Chen, H. Shen, Leveraging single-user applications for multi-user collaboration: the coword approach, in: Proceedings of the 2004 ACM conference on Computer supported cooperative work, ACM, 2004, pp. 162–171.

[7] S. Martin, P. Urso, S. Weiss, Scalable xml collaborative editing with undo, in: OTM Confederated International Conferences On the Move to Meaningful Internet Systems, Springer, 2010, pp. 507–514.

[8] L. Gao, F. Yu, Q. Chen, N. Xiong, Consistency maintenance of do and undo/redo operations in real-time collaborative bitmap editing systems, Cluster Computing 19 (1) (2016) 255–267.

[9] C. Sun, H. Wen, H. Fan, Operational transformation for orthogonal conflict resolution in real-time collaborative 2d editing systems, in: Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work, ACM, 2012, pp. 1391–1400.

[10] Y. Wu, F. He, D. Zhang, X. Li, Service-oriented feature-based data exchange for cloud-based design and manufacturing, IEEE Transactions on Services Computing 11 (2) (2018) 341–353.

[11] X. Lv, F. He, Y. Cheng, Y. Wu, A novel crdt-based synchronization method for real-time collaborative cad systems, Advanced Engineering Informatics 38 (2018) 381–391.

[12] X. Lv, F. He, X. Yan, Y. Wu, Y. Cheng, Integrating selective undo of feature-based modeling operations for real-time collaborative cad systems, Future Generation Computer Systems 100 (2019) 473–497.

[13] W. Shen, J. Barthès, J. Luo, Computer supported collaborative design: Technologies, systems, and applications, Contemporary Issues in Systems Science and Engineering (2015) 537–573.

[14] C. A. Ellis, S. J. Gibbs, Concurrency control in groupware systems, in: Acm Sigmod Record, Vol. 18, ACM, 1989, pp. 399–407.

[15] C. Sun, X. Jia, Y. Zhang, Y. Yang, D. Chen, Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems, ACM Transactions on Computer-Human Interaction (TOCHI) 5 (1) (1998) 63–108.

[16] G. Oster, P. Urso, P. Molli, A. Imine, Data consistency for p2p collaborative editing, in: Proceedings of the 2006 20th anniversary conference on Computer supported cooperative work, ACM, 2006, pp. 259–268.

[17] N. Agustina, C. Sun, Dependency-conflict detection in real-time collaborative 3d design systems, in: Proceedings of the 2013 conference on Computer supported cooperative work, ACM (2013), 715–728.

[18] C. Sun, S. Xia, D. Sun, D. Chen, H. Shen, W. Cai, Transparent adaptation of single-user applications for multi-user real-time collaboration, ACM Transactions on Computer-Human Interaction 13 (4) (2006) 531–582.

[19] A. H. Davis, C. Sun, J. Lu, Generalizing operational transformation to the standard general markup language, in: Proceedings of the ACM 2002 conference on Computer Supported Cooperative Work, ACM, 2002, pp. 58–67.

[20] N. Agustina, C. Sun, Operational transformation for real-time synchronization of shared workspace in cloud storage, in: Proceedings of the 19th International Conference on Supporting Group Work. ACM,2016, pp. 61-70.

[21] J. Day-Richter, What's different about the new Google Docs: Making collaboration fast. DOI: https://drive.googleblog.com /2010/09/whats-different-about-new-google-docs.html

[22] N. Gu, J. Yang, Q. Zhang, Consistency maintenance based on the mark & retrace technique in groupware systems, in: Proceedings of the 2005 international ACM SIGGROUP conference on Supporting group work, ACM, 2005, pp. 264–27

[23] H. Xia, T. Lu, B. Shao, G. Li, X. Ding, N. Gu, A partial replication approach for anywhere anytime mobile commenting, in: Proceedings of the 17th ACM conference on Computer supported cooperative work & social computing, ACM, 2014, pp. 530–541.

[24] N. Gu, Q. Zhang, J. Yang, et al, Dcv: a causality detection approach for large-scale dynamic collaboration environments, in :Proceedings of the 2007 international ACM conference on Supporting group work. ACM,2007, pp.157–166.

[25] L. Gao, W. Tang, High Efficient Consistency Maintenance Strategy of Real-time String Text Editing Systems, International Journal of Hybrid Information Technology 8 (10) (2015) 383–394.

[26] M. Shapiro, N. Preguica, C. Baquero, M. Zawirski, A comprehensive study of convergent and commutative replicated data types, Tech. rep., Technical Report inria-00555588, INRIA (2011).

[27] H. Roh, M. Jeon, J. Kim, J. Lee, Replicated abstract data types: Building blocks for collaborative applications, Journal of Parallel and Distributed Computing 71 (3) (2011) 354–368.

[28] X. Lv, F. He, W. Cai, Y. Cheng, A string-wise CRDT algorithm for smart and large-scale collaborative editing systems, Advanced Engineering Informatics 33 (3) (2017) 397–409.

[29] X. Lv, F. He, W. Cai, Y. Cheng, Supporting selective undo of string-wise operations for collaborative editing systems, Future Generation Computer Systems 82 (2018) 41–62.

[30] X. Lv, F. He, W. Cai, Y. Cheng, An optimized RGA supporting selective undo for collaborative text editing systems, Journal of Parallel and Distributed Computing 132 (2019) 310–330.

[31] X. Lv, F. He, Y. Cheng, Y. Wu, A novel CRDT-based synchronization method for real-time collaborative CAD Systems, Advanced Engineering Informatics 38 (2018) 381–391.

[32] C. Sun, D. Sun, A. Ng, W. Cai, B. Cho, Real Differences between OT and CRDT under a General Transformation Framework for Consistency Maintenance in Co-Editors, in :Proceedings of the ACM on Human-Computer Interaction, ACM, 2020, pp.1–26.

[33] P. A. Bernstein, N. Goodman, Concurrency control in distributed database systems, ACM Computing Surveys 13 (2) (1981) 121–157.

[34] C. Papadimitriou, The theory of database concurrency control, Computer Science Press, Inc. (1986)

[35] M. J. Carey, M. Livny. Distributed concurrency control performance: A study of algorithms, distribution, and replication. University of Wisconsin-Madison Department of Computer Sciences (1988) 1–30.

[36] Y. Chen, F. He, H. Li, D. Zhang, Y. Wu, A full migration bbo algorithm with enhanced population quality bounds for multimodal biomedical image registration, Applied Soft Computing (2020) 106335.

[37] C. Wu, L. Li, C. Peng, Y. Wu, C. Lee, Design and analysis of an effective graphics collaborative editing system, EURASIP Journal on Image and Video Processing 2019 (50) (2019).

[38] Y. Pan, F. He, H. Yu, A correlative denoising autoencoder to model social influence for top-n recommender system, Frontiers of Computer science 14 (3) (2020) 1–13.

Authorized licensed use limited to: Southwest University. Downloaded on April 04,2026 at 08:58:37 UTC from IEEE Xplore. Restrictions apply.